

Wings of Ash — Game Design & Technical Document

Game concept and implementation



Student Name: W D R Pradeepa

IT Number: IT22033482

Module / Course: SE4031

Submission Date: 28/03/2028



SLIIT UNI

THE KNOWLEDGE UNIVERSITY

Table of Contents

1. Summary of the Game Concept	2
2. Description of Core Features	3
2.1 Health System (“Flame Health”)	2
2.2 Score System	3
2.3 Health Packs (“Ember Hearts”)	3
2.4 Projectile Attacks (“Fire Breath”)	4
2.5 Enemies (“Shadow Drakes”)	4
2.6 World Scroll & Speed	4
2.7 Spawning System	5
2.8 UI Flow	6
2.9 Player Movement	6
3. Screenshots of Gameplay	7
4. Control Guide	8
5. Creative Feature — Rage Flame Mode	9
6. Credits — External Assets & Tools	10
6.1 Unity Asset Store (UI Assets)	10
6.2 Visual Elements	11
6.3 Tools & Technologies	11
7. Video Link (Submission Requirement)	11
8. Zipped Folder Link (Submission Requirement)	11
9. Zipped Windows .exe Build Folder Link	12
10. Appendix — Technical References	12

1. Summary of the Game Concept

Wings of Ash is a 2D endless side-scrolling game. The player controls a young dragon carrying the last sacred ember of a fallen sky kingdom. The dragon flies through ruined gates and stormy skies while shadow drakes and hazards threaten the ember.

Core Fantasy:

- Survive as long as possible
- Score by passing through gate openings
- Collect ember hearts to restore health
- Use fire breath to destroy enemies

Difficulty increases as the world scroll speed ramps up over time.

Win/Lose Conditions:

- The run ends when the player runs out of lives
- Each life starts with full health
- Obstacles may cause instant death
- Score persists across lives during a run
- High score is stored locally

2. Description of Core Features

2.1 Health System (“Flame Health”)

- Implemented using a FlameHealth component
- UI displays:
 - Health bar (slider)
 - Text (*Health: current / max*)

Mechanics:

- Collisions with obstacles/enemies reduce health
- Some obstacles may cause instant death
- Multiple lives system:
 - Respawn countdown (e.g., 3–2–1)
 - Full health restored on respawn
- Game Over screen shown when lives reach zero

2.2 Score System

- Managed by ScoreManager

Scoring Rules:

- +1 → Passing through a gate
- +5 → Destroying an enemy
- Score may persist across lives
- Resets when restarting the game

2.3 Health Packs (“Ember Hearts”)

- Spawned using GameplaySpawner
- Use trigger colliders

Behavior:

- Restore health up to maximum
- Destroyed upon collection
- Optional spawn validation to avoid overlap

2.4 Projectile Attacks (“Fire Breath”)

- Triggered via left mouse click
- Spawned from a fire point (dragon mouth)

Projectile Behavior:

- Moves forward
- Destroys on hit or timeout
- Ignores score triggers (passes through gates)
- Rendered above background layers

2.5 Enemies (“Shadow Drakes”)

- Controlled by ShadowDrakeEnemy

Features:

- Move with world scroll
- Damage player on collision
- Can be destroyed by player projectiles
- May shoot projectiles at player

Difficulty Scaling:

- Spawn rate increases over time

2.6 World Scroll & Speed

- Managed by WorldScroller

Behavior:

- Moves all elements left continuously
- Speed increases gradually with a cap
- Supports parallax backgrounds

2.7 Spawning System

Handled by GameplaySpawner:

- Spawns:
 - Gates
 - Obstacles
 - Enemies
 - Health pickups

Features:

- Timer-based spawning
- Random variation (jitter)
- Distance constraints
- Optional collision checks

2.8 UI Flow

Flow sequence:

Loading → Main Menu → Main Game

Main Menu Options:

- Play
- Story
- How to Play
- Settings
- Exit

Pause Menu:

- Resume
- Restart
- Main Menu

2.9 Player Movement

- Flappy-style control

Controls:

- Space → Upward impulse
- Gravity pulls player downward
- Optional vertical bounds

3. Screenshots of Gameplay



Figure 1: Game Paused screen

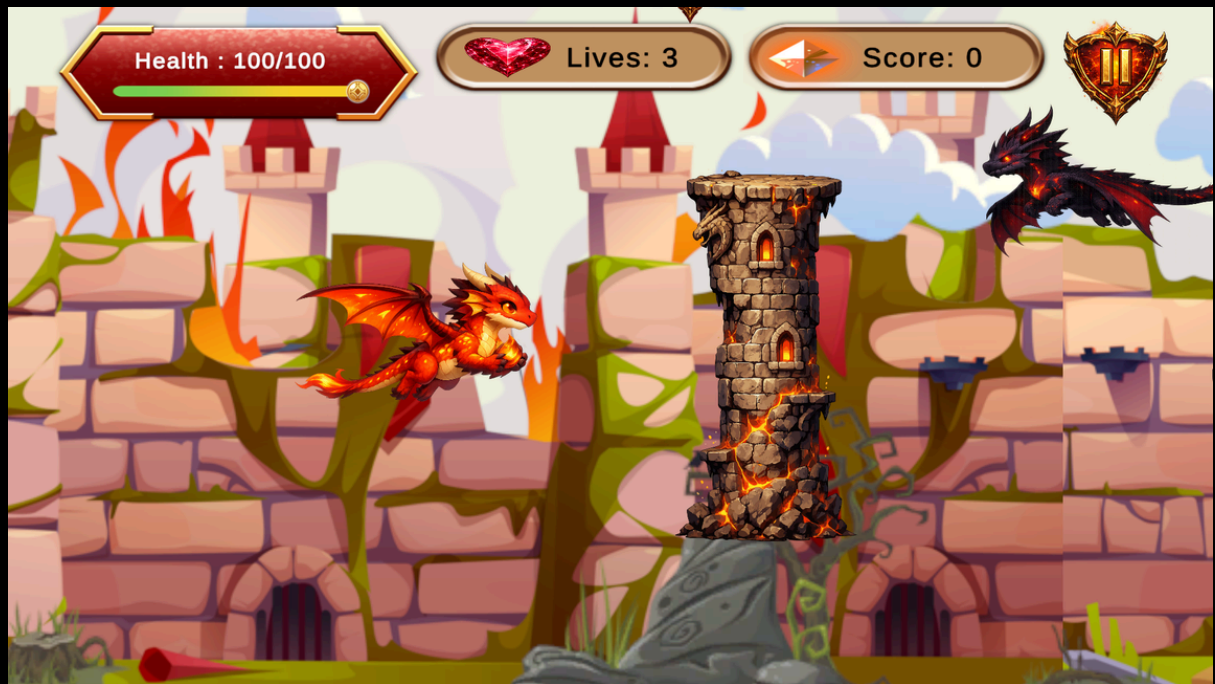


Figure 2: Max Health Gameplay](path-to-image)



Figure 3: Countdown after player death with red vignette overlay



Figure 4: Low Health Gameplay](path-to-image)

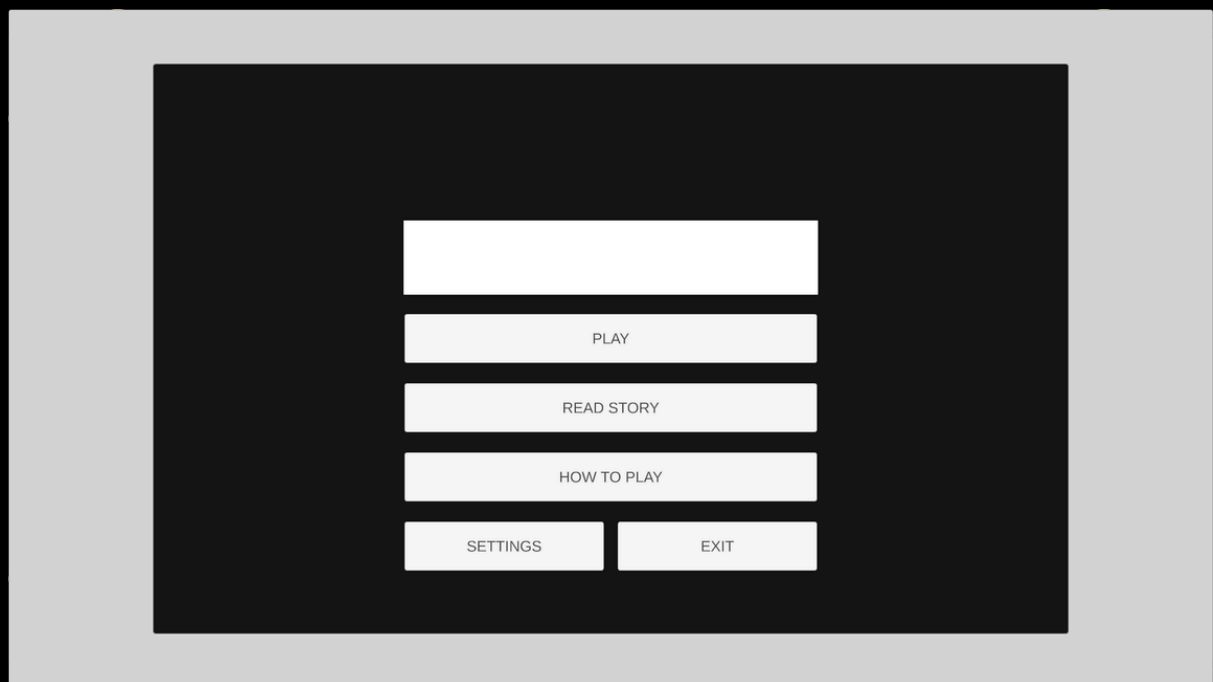


Figure 5: Main menu using custom UI layout

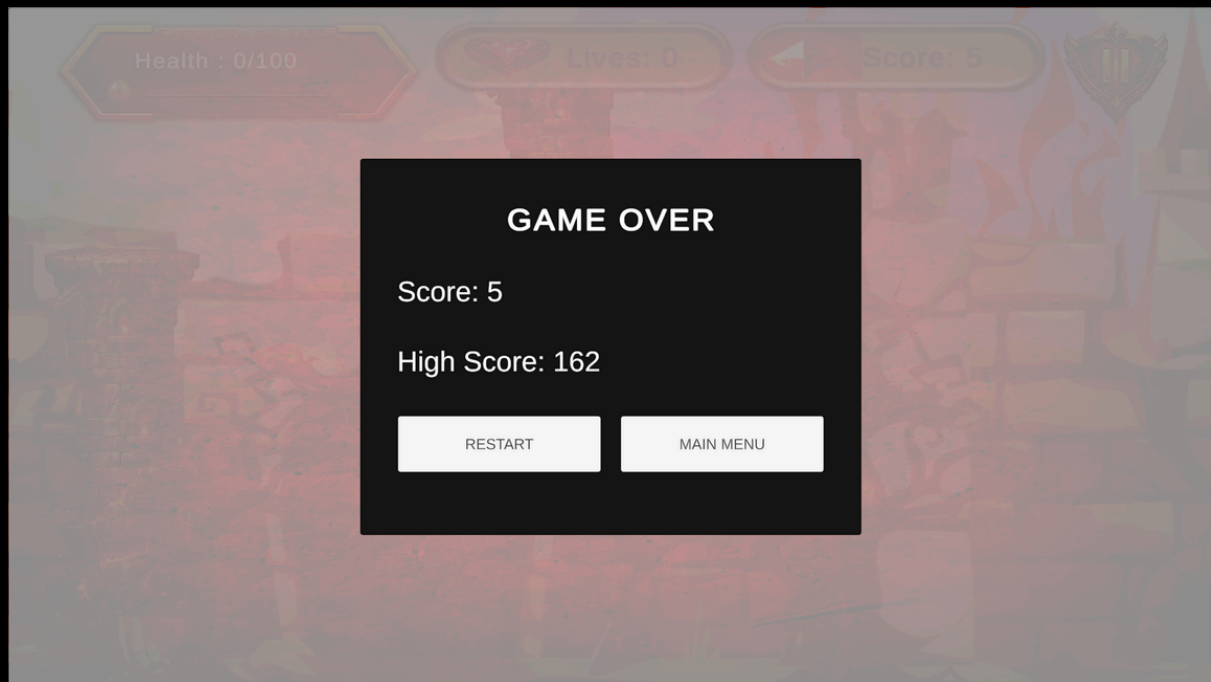


Figure 6: Game Over Screen

4. Control Guide

Action	Input
Fly Up (Flap)	Space
Fire Attack	Left Mouse Button
Pause	UI Button / Key
Navigate Menus	Mouse

5. Creative Feature — “Rage Flame Mode”

Purpose:

Enhance player feedback and intensity when health is low.

Implementation:

- Reads normalized health value
- Health bar shifts:
 - Gold → Red

Effects:

- Screen vignette increases (dark red overlay)
- Camera shake on damage
- Fire breath becomes stronger visually
- Background tint intensifies

Outcome:

- Creates a feeling of urgency and power
- Enhances immersion without altering core mechanics

6. Credits — External Assets & Tools

6.1 Unity Asset Store (UI Assets)

Asset	Publisher	Usage
Fantasy Free GUI	ZOSMA	UI elements, menus, HUD

License: Unity Asset Store standard license

6.2 Visual Elements

Source	Description
Freepik	Base artwork / references
ChatGPT / DALL·E	Generated assets (dragon, effects, backgrounds)

6.3 Tools & Technologies

Tool	Purpose
Unity (v6.x)	Game engine
C#	Game scripting
TextMeshPro	UI text
Visual Studio / Rider	Development IDE

7. Video Link

https://drive.google.com/file/d/1K9E3UxhKJ16pwt8HfIFwe-ZqNpQwbpJ7/view?usp=drive_link

8. Zipped Unity Project Folder Link

https://drive.google.com/file/d/1GShd9Qe0Re057CHcDqYQeLKRU0XXBWF5/view?usp=drive_link

9. Zipped Windows .exe Build Folder Link

https://drive.google.com/file/d/1nLxXPoGl7T6BzBxK6vj1qjdqg0pkWw7x/view?usp=drive_link

10. Appendix — Technical References

Scenes:

- 00_Loading
- 01_MainMenu
- MainGame

Key Scripts:

- PlayerFlightController
- PlayerShoot
- FlameHealth
- ScoreManager
- WorldScroller
- GameplaySpawner
- ShadowDrakeEnemy
- GamePauseManager
- RageFlameMode
- CameraFollow
- CameraShake